

3. Árboles

Árboles de decisión

Nosotros tenemos una función f tal que:

$$f : X \rightarrow Y$$

y lo queremos ajustar a $h_{\theta} : X \rightarrow Y$ para que, a partir de:

$$\mathbb{D} = \{(x^1, y^1), \dots, (x^m, y^m)\}$$

para conseguir lo siguiente:

$$\begin{array}{c} x^1 \\ x^2 \\ \vdots \\ x^m \end{array} \begin{bmatrix} x_1^1 & x_2^1 & \dots & x_n^1 \\ x_1^2 & x_2^2 & \dots & x_n^2 \\ \vdots & \vdots & & \vdots \\ x_1^m & x_2^m & \dots & x_n^m \end{bmatrix} \begin{bmatrix} y^1 \\ y^2 \\ \vdots \\ y^m \end{bmatrix}$$
$$f^1 \quad f^2 \quad \dots \quad f^n \quad y$$

Podemos representar cualquier función de los atributos de input por medio de *árboles de decisión*. Modela los datos que tenemos. Entonces, ¿cómo se crean? Imagina que lo siguiente es una función.

1. Verificamos si todos los datos que tenemos pertenecen a la misma clase, no hay datos o no hay características.
2. Escogemos una característica de estos datos.
3. Separamos los datos que contiene esta característica.
4. Por cada valor que tiene nuestra característica, creamos un nodo.
5. Luego creamos otro hijo usando el input del árbol actual por medio de recursión (1).
6. Esto se repite hasta que completamos nuestro árbol, cuando se nos acaben nuestra características.

Entre menos características necesite un modelo para explicar un problema, mejor. Entonces generamos todos los árboles posibles y nos quedamos con el árbol mas pequeño/corto.

Existe un problema: Si tenemos n variables y cada variable tiene 2 valores, la cantidad de árboles, en el *peor* de los casos es 2^{2^n} árboles. Aquí podemos usar un algoritmo que utilice cierta heurística que nos permita confirmar que el árbol es el más pequeño.

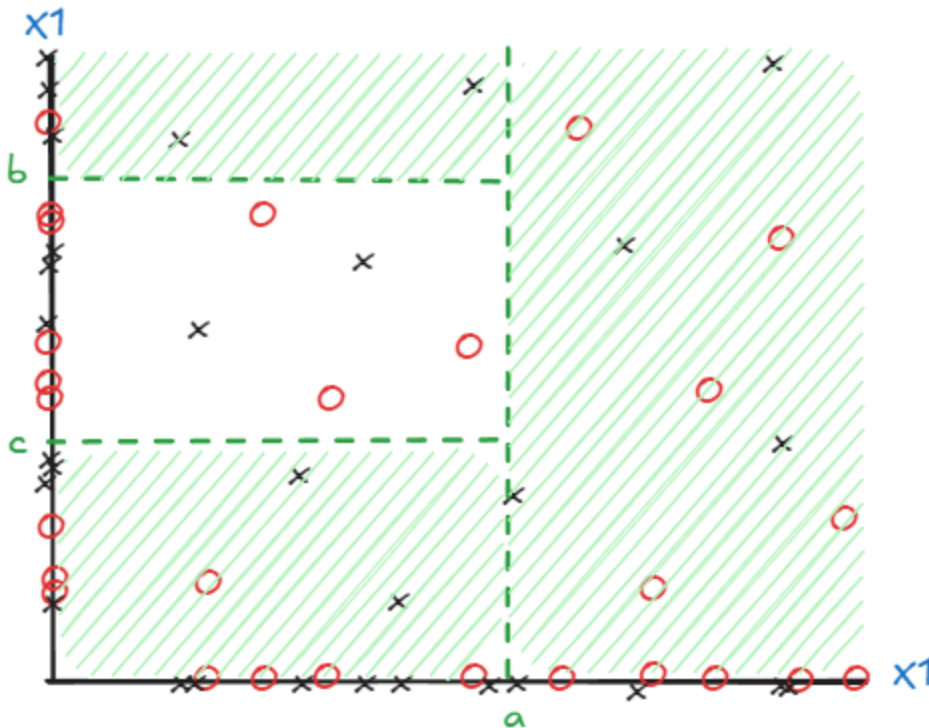
El truco está en decidir cuál atributo elegimos. No hay una manera de medir de manera numérica que tan bien agrupan los datos, pero podemos hacer uso de la *Entropía* (del valor de salida) para medir que tan desordenado esta el nodo actual y sus hijos.

Los árboles en general... no son muy buenos para una gran cantidad de datos (Big Data) pero llega a servir. A este algoritmo se le llama **ID3**.

Tratamos de no sobre-aprender si no que regularizamos.

Pero, ¿qué pasa si tenemos árboles que se rigen por valor y no por característica? Podemos hacer dos cosas diferentes.

1. Discretizar mis valores (binning).
2. Hacer árboles binarios en donde seleccionamos en dos valores (mayor que, menor que).



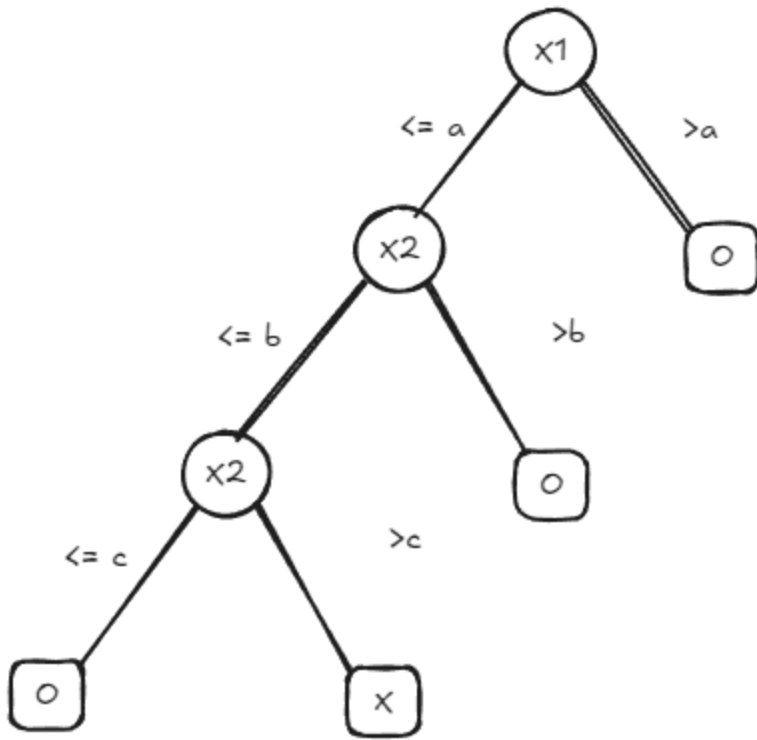
Queremos particionar (a) los ejes x_1 y x_2 en las partes en donde los valores cambian (o van de x a a o van de a a x) para así obtener la mayor ganancia de información y separar los datos por clase.

Esto se puede hacer con la siguiente ecuación.

$$G(y) = 1 - H(y) = 1 - \frac{1}{n} \sum_{i=1}^n \log_2 \left(\frac{n}{n_i} \right)$$

$$G(y, a) = 1 - \frac{1}{n} \left(\sum_{i=1}^n \log_2 \left(\frac{n}{n_i} \right) + \sum_{j=1}^m \log_2 \left(\frac{n}{n_j} \right) \right)$$

Así se puede hacer un árbol binario.



Ejemplos

Imaginate que tienes un puzzle de bloquitos en donde los puedas mover si hay un espacio vacío cerca:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	○

○ = Celda Vacía

Para esto podemos describir un problema de búsqueda:

S_{start} = Startin State

Actions(s) : ossible Actions

Cost(s, a) : Action cost $\rightarrow \{p, on, uierda, ereca\}$

Succ(s, a) : Succesor

sEnd(s) : Reaced end state

```

class NPuzzle(SearchPb): # Define el problema del NPuzzle inicial. Da la
configuracion inicial
    def __init__(self, n, Sf = None)
    self.n = n
    if Sf != None and len(Sf) != n ** 2:
        raise ValueError('Sf y n no coinciden')
    if Sf == None:
        self.S_f = [range(n ** 2)]
    elif sorted(Sf) != [range(n ** 2)]:
        raise ValueError('Sf no es una permutación')
    else:
        self.S_f = Sf[:]

    def acciones(self, s):
        if not exist(self.a_legales):
            self.a_legales = {}
            for i in range(self.n ** 2):
                temp = []
                if i // self.n > 0:
                    temp.append('Up')
                if i // self.n < self.n - 1:
                    temp.append('Down')
                if i % self.n > 0:
                    temp.append('Derecha')
                if i % self.n < self.n - 1:
                    temp.append('Izquierda')
                self.a_legales[i] = temp[:]

            return self.a_legales[s.index[0]]

    def sucesor(self, s, a):
        costo_local = 1

        if a not in self.acciones(s):
            raise ValueError('___')

        s_n = s[:]
        i_vacio = s_n.index(0)
        i_otro = (i_vacio + 1 if a == 'Derecha' else
                  i_vacio - 1 if a == 'Izquierda' else
                  i_vacio + self.n if a == 'Down' else
                  i_vacio - self.n)
        s_n[i_vacio], s_n[i_otro] = s_n[i_otro], s_n[i_vacio]

        return s_n, costo_local

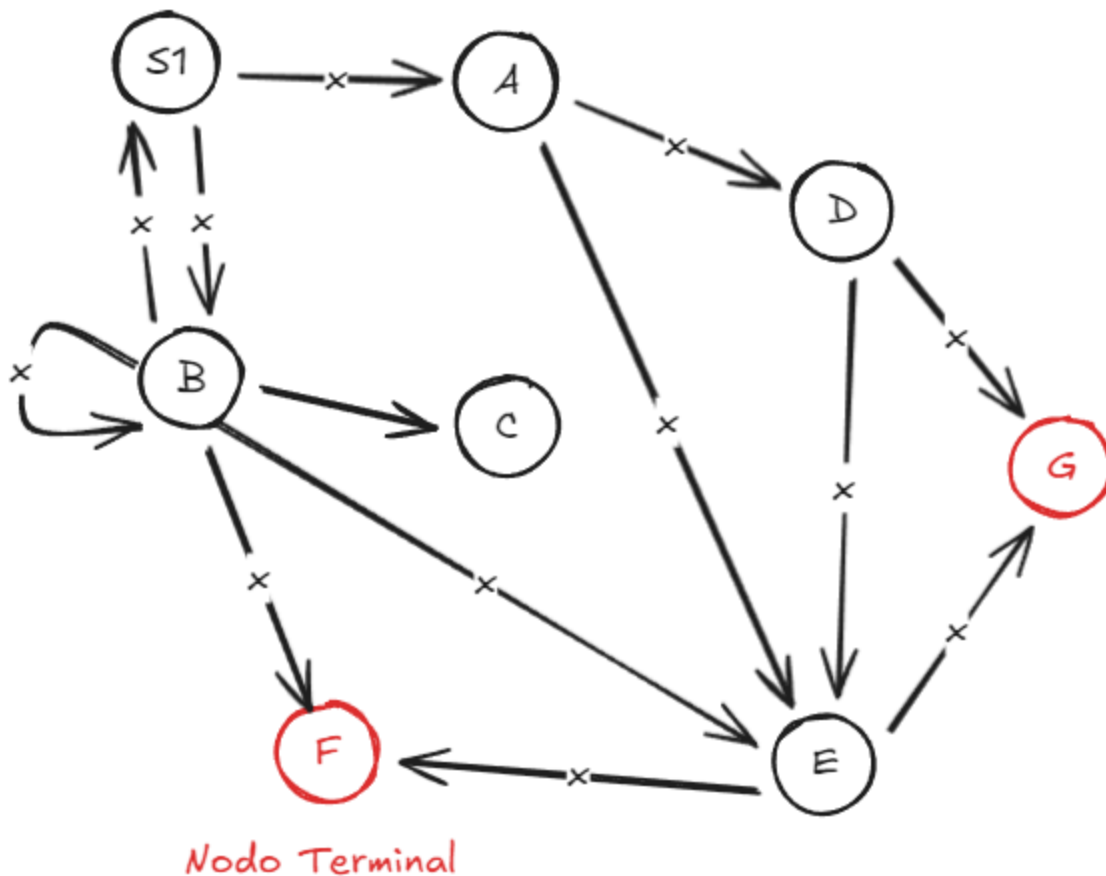
```

```
def terminal(self, s):
    return s == self.s_f
```

Problemas que se pueden resolver por búsqueda

Un grafo se puede resolver si:

- Tiene nodos terminales.
- Tiene costos de movimiento.
- Se conoce el estado inicial.
- Se pueden tomar acciones. $A(s) = \text{Acciones}(s)$
- Se pueden calcular los sucesores en base al estado actual y el costo de movimiento.
 $s_{n+1} = \text{seer}(s, a)$
- Se conocen nodos terminales. $\text{terminal}(s)$

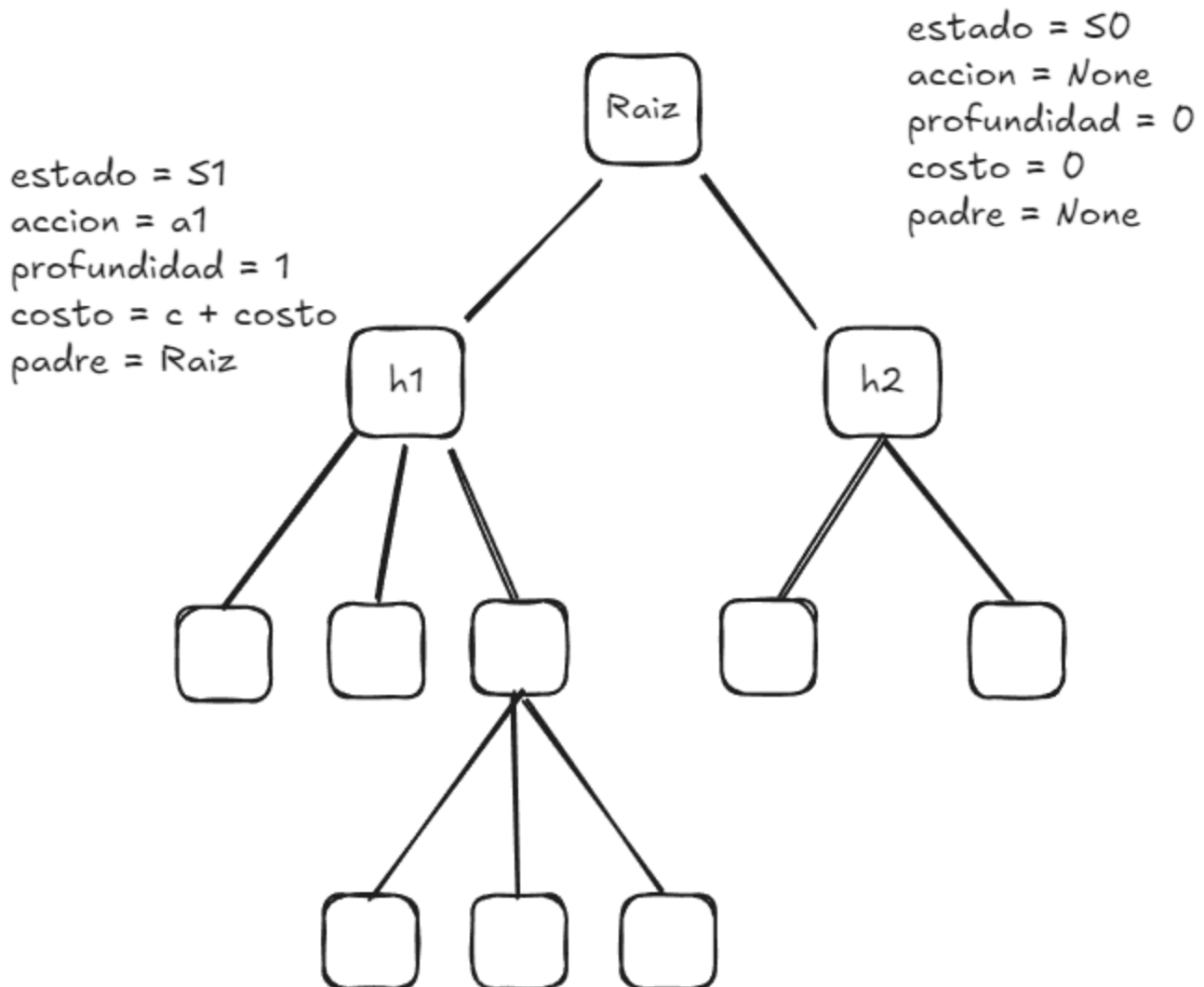


Estos grafos se pueden traducir a árboles de decisión. Se tiene un nodo raíz cuyo el cual donde:

- estado = S
- acción = None
- costo = 0

- profundidad = 0
- Padre = None

Ve los nodos como estructuras con sus propios datos.



Vamos a hacer un programa genérico para árboles:

```
class NodoSearch(object):
    # Clase de nuestro nodo

    def __init__(self, s, a = None, padre = None, costo_l = 0):
        # Define el nodo y sus atributos
        self.s = s # Define el estado
        self.a = a # Define la acción que te llevo a este nodo
        self.padre = padre # Define el nodo padre

        self.d = 0 if padre == None else self.padre.d + 1
        # Si este nodo es raiz, entonces la profundidad es 0, si no, toma el
        # depth del nodo padre y le suma 1
```

```

        self.costo_l = 0 if padre == None else costo_local +
self.padre.costo_l
        # Calcula el costo que se requiere para llegar a este nodo

def expande(self, search_pb):
    # search_pb es "Search Problem"
    # Calcula el siguiente nodo

    for a in search_pb.acciones(self.s):
        # Por cada una de las acciones que tengas...

        s_n, costo_l = search_pb.sucesor(s,a)
        # Generas el siguiente nodo y su costo.

        yield NodoSearch(s_n, a, self, costo_l)
        # yield es como return pero guarda los procesos (la función se
suspende)
        # Aquí el nodo va regresar un hijo y luego el siguiente y el
siguiente...
        # Así la memoria no se sobre carga horriblemente.

def plan(self):
    # Genera una lista de todos los nodos pr los cuales se tienen que pasar
para
    # llegar al nodo que llamó la función.

    return [(self.s, None, None)] if self.padre == None else
self.padre.plan()[::-1] + [(self.padre.s, self.a, self.cost), (self.s,
None, None)]

    # Si es el nodo raiz, regresamos la lista con el "primer paso".
    # Si ejecutamos esta función en cualquier otro nodo. Se llama
    # recursivamente, generando un plan (o camino) mediante la lista que
    # describe como llegas al nodo que llamó la función, empezando por el
nodo

    # raiz (s1, a1, c1) y enlistando todos los demas nodos con sus
acciones y
    # costos que se requirieron para cada movimiento hasta llegar al nodo
    # buscado (self.s, None, None). Por eso empezamos en el indice [::-1]
para
    # no incluir un nodo indeseado.

def busqueda_generica(s, pb): # pb es problema

frontera = [NodoSearch(s)]

```

```

# La frontera se inicializa

while frontera:
    # Mientras la lista no este vacía...

    plan = saca_nodo(frontera)
    # Sacamos un nodo de la frontera

    if pb.terminal(plan.s):
        # Si encuentra un nodo terminal

        return plan
        # Regresa el plan

    for plan_hijo in plan.expande(pb):
        agrega_frontera(frontera, plan_hijo)

return None
# No existe plan posible para llegar al nodo que buscamos

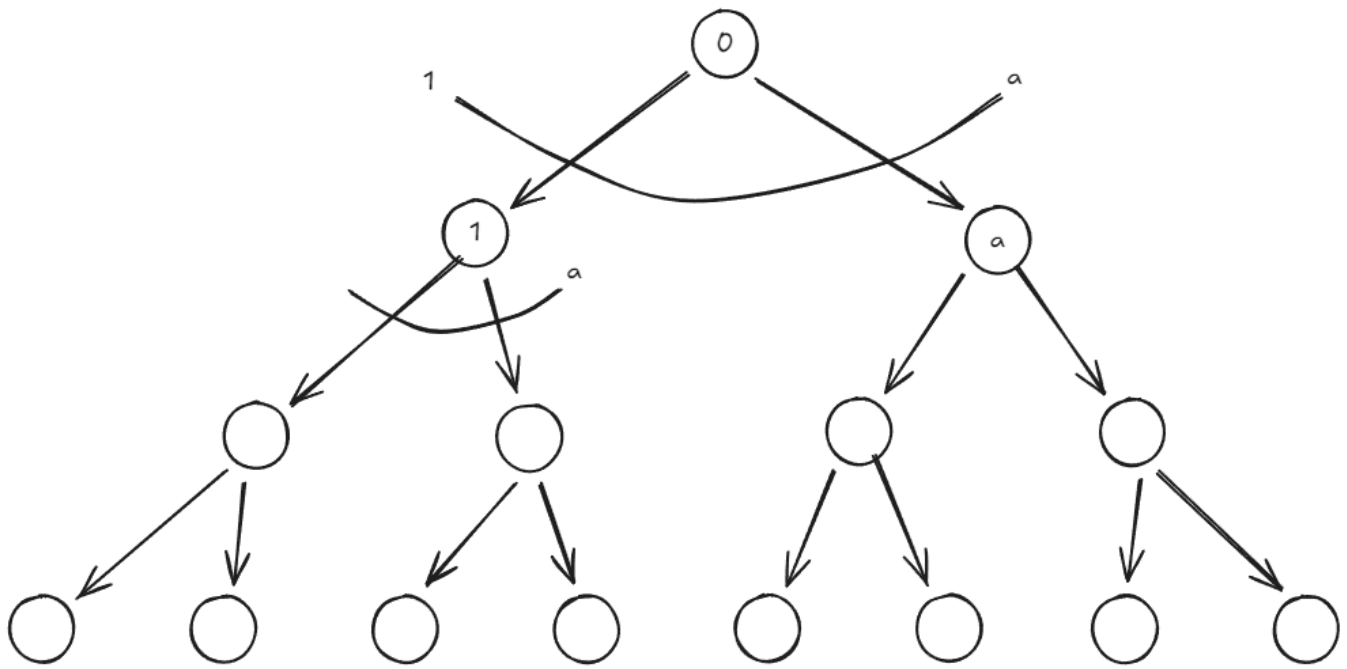
```

Si llegamos a un plan terminal (o nodo terminal) en donde se encuentra el menor costo y profundidad, decimos que encontramos c y d respectivamente. Si encontramos el mayor costo y profundidad en el nodo terminal, encontramos c y d respectivamente.

Para decir que un algoritmo es optimo, tiene o es:

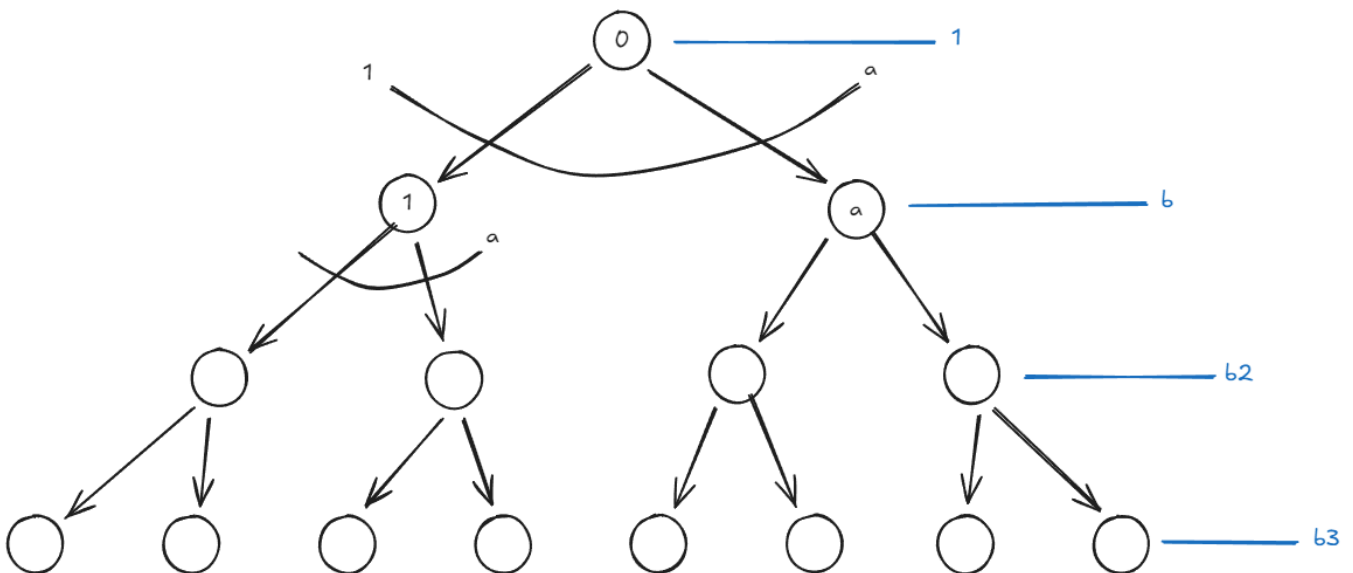
- **Admisible:** Si existe una o varias soluciones al menos encuentra una.
- **Optimalidad:** El algoritmos es admisible y la solución que encuentra es la solución de menor costo.
- **Complejidad temporal:** Entre más crece el problema, toma más tiempo en resolver. (2^n , donde n es el número de parámetros que acepta el algoritmo).
- **Complejidad material:** Entre más crece el problema, más memoria consume el algoritmo.

Cuando estas en un nodo (o estado) vas a generar nuevos planes. Asumiendo que tienes a nodos, puedes generar de 1 a a planes. Si inicializamos un pila vacía $\square$ podemos ir poniendo todos los planes que hemos tomado.



$$\begin{aligned}
 & \square \\
 & \square \rightarrow [1, \dots, a] \\
 & [2, \dots, a] \rightarrow [2, \dots, a, 11, \dots, 1a] \\
 & [\dots, a, 11, \dots, 1a] \rightarrow [\dots a, 11, \dots, 1a, 21, \dots, 2a]
 \end{aligned}$$

Así vamos a ir hasta llegar a un plan valido que nos lleve a nuestro nodo terminal. Pero, ¿cómo vamos a medir el tiempo? Podemos verlo de la siguiente manera:



Si el plan admisible de menor profundidad tiene profundidad d entonces vamos a revisar:

$$1 \quad 2 \quad d \quad \text{Esto es: } ()$$

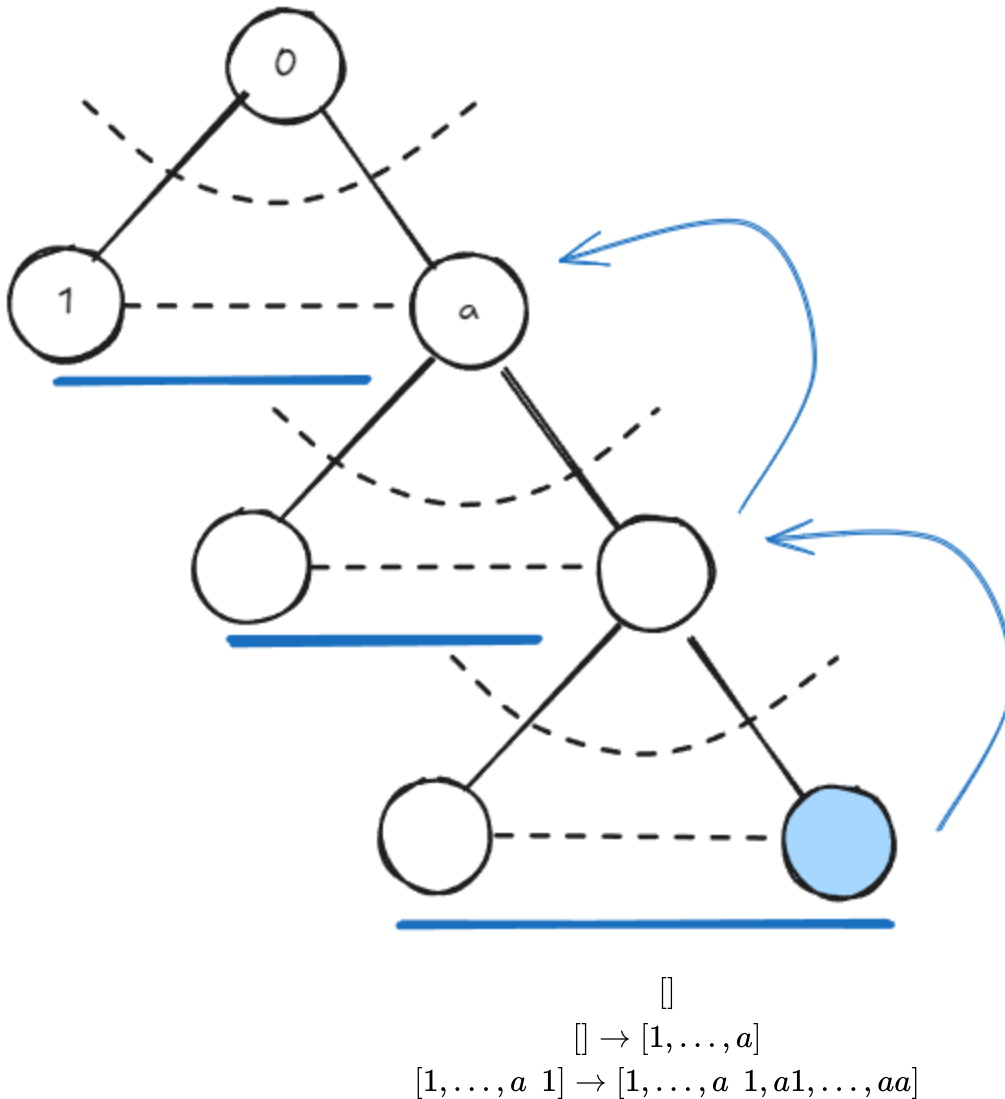
Este algoritmo se le llama "Primero a lo ancho".

- ¿Es admisible? Si, eventualmente llegamos a una respuesta.

- ¿Es *optimo*? Si, solo si el costo local sea equivalente a 1 o siempre y cuando estemos buscando el plan con menos pasos.
- Su *complejidad temporal* es: $(^d)$
- Su *complejidad material* es: $(^{d1})$

Es más complicado para este modelo la complejidad material que la temporal, debido al tamaño de memoria requerido.

Ahora vamos a probar con una *pila*.



A este algoritmo se le llama *Depth First Searching*. Se inicia desde abajo y va revisando hacia arriba.

- ¿Es *admisible*? No, si el "depth" máximo es .
- ¿Es *optimo*? No podemos asegurar nada.
- Su *complejidad temporal* es: $(^{d\text{ ma}})$

- Su complejidad material es: (d_{max})

Ejemplo

Tienes calles con manzanas numeradas de 1 a n . Al caminar de s a $s+1$ se tarda 1 minuto. Si tomas un tren mágico de s a $2s$ toma 2 minutos. ¿Cómo me traslado de 1 a n en el menor tiempo?

Si tomas el tranvía primero y luego caminas (considerando de que tu estado final es 1) pasa lo siguiente:

posición	minutos para el próximo estado
1	2
2	2
	2
	2
1	2
2	2
	2
12	22
⋮	
1	total:

Si una combinación del tranvía y caminata (considerando de que tu estado final es 1) pasa lo siguiente:

posición	minutos para el próximo estado
1	1
2	2
	2
	1
	2
1	2
	1
	2
	1
	2
1	total: 1

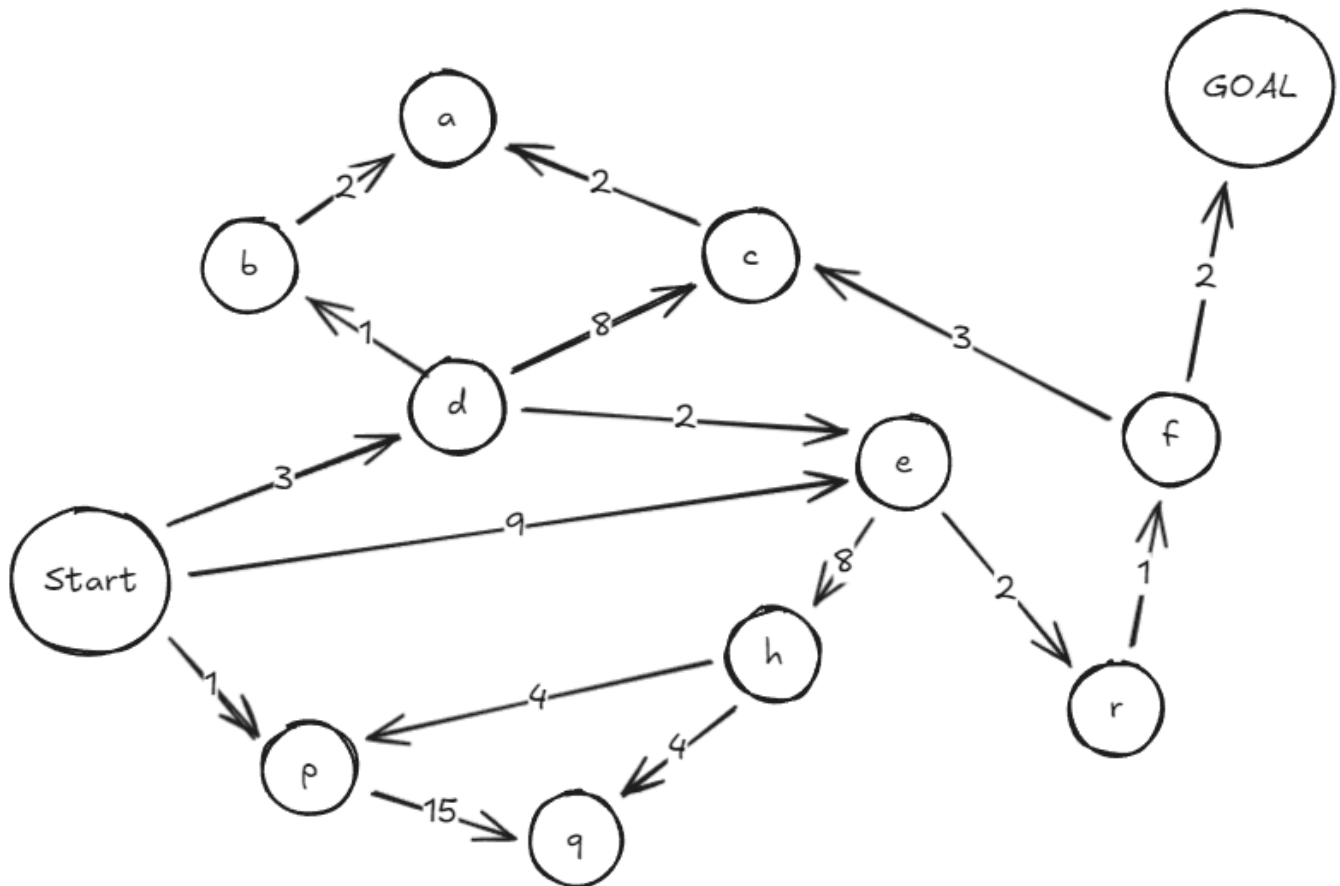
La advertencia de Santayana

"Aquellos que no recuerdan el pasado están condenados a repetirlo"

- *George Santayana*

La falla en detectar estados repetidos pueden generar exponencialmente más trabajo. Para evitar esto, a nuestro criterio de búsqueda le vamos a agregar una variable, el cuál es un conjunto de estados visitados. Así, cuando el programa visite el nodo y vea que ya está visitado, no lo agrega a la frontera.

La búsqueda en grafos es casi siempre mejor que una búsqueda de árboles (en la práctica). Para agregar los costos, lo hacemos mediante pares. Primero el costo del nodo y luego el nodo mismo $(n.\text{costo}, \text{nodo})$.



Como sabemos que este algoritmo siempre es el mejor?

es un plan óptimo.

$\text{terminal}(\text{estado}) == \text{rue}$
 $\text{costo} \leq \text{costo}$

Para todo tal que

$\text{terminal}(\text{estado}) = \text{rue}$

Cuando sacamos p de frontera hemos revisado todos los planes p tal que

$p.\text{costo} \leq \text{costo}$

y algunas que

$p.\text{costo} = \text{costo}$

pero NINGUNO que

$p.\text{costo} < \text{costo}$

Entonces terminamos con que.

$$\begin{pmatrix} \text{costo} \\ \end{pmatrix}$$

Pooooooooor eso, es mejor tener un **IA** basado en *heurística*.

Inteligencia basada en heurística.

Tenemos que $h = \text{eurística}(\text{plan})$. h es un costo estimado de un problema relajado.

h costo de un plan completo desde planestado

h iniciando en planestado

Si esto ocurre $s \in S$ se dice que la heurística es admisible.

Si comparamos los algoritmos S y Greedy...

$\text{norm Cost Searc}(S) \rightarrow (\text{ncosto}, n)$

$\text{reed} \rightarrow (\text{eurística}(n), n)$

Pero que tal si combinamos $(\text{ncosto eurística}(n), n)$

es in plan completo (el estado de es un terminal) que pasa por n donde n es un plan intermedio...

$\text{costo } n \text{ costo eurística}(n)$

$\text{eurística}() =$ Si ya esta en un estado terminal su heurística es 0

Si es un plan OPTIMO...

$\text{eurísitca}() =$

$\text{costo } \text{costo}$

$\text{costo } \text{costo eurística}()$

Este algoritmo se llama $A(n \text{ costo eurística}(n), n)$

La heurística se calculan desde el estado en el que estoy hasta donde me falta. Es importante que mi heurística sea admisible para estar seguros de que encuentre em mejor plan.

h es admisible si $h(n) \leq c(n)$, donde c es el costo del plan óptimo iniciando en n estado. $h(n) = 0$ si n estado es terminal.

Algoritmo A

Usando el *8 Puzzle*:

- Estados: $S = (s, \dots, s)$.
- No. de estados: $S \in \{, \dots, \}$.
- Cantidad de permutaciones: $\text{card}S = 2$
- Acciones: $\{, , \rightarrow, \}$

Nuestras heurísticas pueden ser las siguientes:

h_1 : Número de piezas mal acomodadas.

h_2 : Distancia Manhattan de cada pieza a su posición.

Las heurísticas son menores a la cantidad de pasos que realmente se tienen que dar, lo cual es de esperarse.

Podemos programar toda esta vaina en *Python* (classy).

Los nodos en A ordena los nodos por n costo $h(n)$. Si n estado es terminal

n costo $h(n) = n$ costo. Mientras mi heurística vaya disminuyendo, menos nodos tenemos que visitar.

Heurística Trivial $h(n) = n$

$h(n)$ n costo es el costo exacto de la solución optima en donde n sea un plan intermedio. Si

$h_1(n)$ $h_2(n)$ n entonces h_2 domina a h_1 . Significa que si una heurística es mejor que la otra, nos deshacemos de la otra. *As simple as that*.

Minmax

```
def nega_val(juego, s, j, ordena=None):

    if juego.terminal():
        return j * juego.ganancia(s) * l

    # USAMOS TABLAS DE TRANSPONICION. LO PUEDES PONER EN EL RETURN DE ARRIBA

    if d == 0:
        j * utilidad(s) * e

    if s in TT and TT[s][0] < d:
        j * TT[s][1] * e

    # -----

    jugadas = ordena(list(---))
    v = -infinite

    for a in jugadas:
        v = max(l * v, l * nega_val(s, -j, -beta, - alpha, -l))

    if v >= beta:
        return (l * v, a)

    alpha = max(a, v)
```

```
return l * a
```

Tabla de transposición

Trazas

Búsqueda quiescente